

Python Programming – Day 3

Class Notes & Summary

1. The Best Way to Learn Programming

The session opened with one of the most important mindset tips for any beginner programmer:

"Make mistakes and learn from them. That is how you debug."

💡 **Instructor Note:** *This is absolutely correct, and it's how every professional programmer learns. When you make a mistake and fix it yourself, it sticks in your memory far better than just watching code work. Don't be afraid of errors — they are your best teachers.*

The Tester Story – Why Mistakes Are Good News

A real-world scenario was discussed to drive this point home:

- You write code as a developer. It goes to a testing team.
- If the tester finds bugs — is that good news or bad news?
- Answer: It has TWO sides.
- **It's better to find bugs in testing than in production. Before delivery to the client is the right time to fix things.** Good side:
- **Your performance evaluation is partly based on how many bugs the tester finds in your code.** Bad side:
- **If your code breaks in production (after delivery to the client), it comes back to you. You cannot escape it — whoever wrote the code is responsible.** Worst case:

💡 **Instructor Note:** *This story teaches a crucial industry reality: always test your own code before submitting it. Companies lose money and reputation when production systems fail. As a developer, your code is your signature.*

2. Running a Python File

Before diving into concepts, the instructor showed how to run a Python file from the terminal.

Command

```
python filename.py
```

Example: If your file is named test.py:

```
python test.py
```

Where Does Execution Start?

- If the file has no classes, objects, or functions — execution starts from line 1 and goes line by line.
- If functions/classes are present — execution starts from the main function (we will cover this later).
- Python uses an interpreter, so it executes line by line. If there is an error on line 2, it will stop there and will not go to line 3.

3. Syntax and Syntax Errors

What is Syntax?

Students gave answers like "format", "set of rules", "structure". The instructor confirmed:

Syntax = the set of rules/design of a programming language that you MUST follow when writing code.

- Different languages have different syntax. C uses printf, Python uses print, Java has its own style.
- Python's syntax is fixed by design — you cannot go in a different direction.
- If you break the syntax rules, Python throws a Syntax Error.

Syntax Error – Example

When writing print(hello world) without quotes, Python throws:

```
SyntaxError: invalid syntax
```

Why? Because Python's rule (syntax) says strings must be inside double or single quotes.

```
print("hello world")    # Correct
print(hello world)      # Wrong - SyntaxError
```

 **Instructor Note:** When you see 'SyntaxError: perhaps you forgot a comma' — Python is just giving you a suggestion. The error message is pre-written inside the Python package and shown when you break a rule. Read the error message carefully — it almost always tells you what went wrong.

Types of Errors (introduced in this session)

SyntaxError	You broke Python's grammar rules. Example: missing quotes, missing brackets.
NameError	You used a variable name that was never defined. Python is case-sensitive: 'a' and 'A' are different variables.
TypeError	You tried to combine incompatible types. Example: string + integer without type conversion.
ValueError	You tried to convert something invalid. Example: <code>int("hello")</code> — "hello" is not a number.

💡 **Instructor Note:** Every error message is your friend. Don't panic when you see red text. Read it carefully — it tells you the type of error and the line number. Fix one error at a time, starting from the top.

4. Variables

What is a Variable?

A variable is a named location in memory (RAM) that stores a value.

- When you write:

```
a = "welcome aboard"
```
- What happens internally: A space is allocated in RAM. That space is labeled 'a'. The value 'welcome aboard' is stored there.

To print the value stored in a variable:

```
print(a)
```

💡 **Instructor Note:** Think of a variable like a labeled box. The variable name is the label, and the value is what's inside the box. Whenever you need that value, you just refer to the label.

Memory – RAM vs Disk

- The .py file is stored on disk (secondary storage / hard drive).
- When you run the file, execution and memory allocation happen in RAM.

- Actual processing (calculations) happen in the CPU.

Python is Case-Sensitive

Python treats uppercase and lowercase letters as completely different:

```
a = "welcome"
print(a)    # Works
print(A)    # NameError: name 'A' is not defined
```

Always use exactly the same case when declaring and using a variable.

Data Types in Python

In Python, you do not declare the data type yourself. Python decides the data type automatically at runtime (called dynamic typing).

str (String)	a = "hello" → type: str
int (Integer)	a = 10 → type: int
float	a = 10.5 → type: float
bool	a = True → type: bool

To check the data type of any variable, use:

```
print(type(a))
```

Note: No 'char' in Python

In languages like C or Java, a single character is called a 'char'. In Python, there is no separate char type. A string of length 1 is still a string (str).

```
x = "D"    # This is a string of length 1, not a char
```

5. Input Function

The input() function is used to take input from the user at runtime.

```
name = input("What is your name? ")
print("My name is " + name)
```

Important: input() Always Returns a String

No matter what the user types — a number, a symbol, anything — `input()` treats it as a string by default.

```
age = input("What is your age? ")
print(type(age))    # <class 'str'> - even if user types 22
```

 **Instructor Note:** *This is a very common source of bugs for beginners! If you expect a number from the user and try to do math with it, you will get a `TypeError`. Always be aware of the type returned by `input()`.*

6. String Concatenation

Concatenation means joining two or more strings together. In Python, we use the `+` symbol:

```
"My name is " + name + " and I am " + age + " years old."
```

TypeError: Cannot Concatenate String and Integer

You cannot use `+` to join a string and a number directly:

```
print("Random " + 10)    # TypeError!
```

This is because `+` between a string and an int is ambiguous — should it add them as numbers or join them as text? Python says: decide clearly.

7. Type Conversion

Type conversion is converting a value from one data type to another.

Integer to String

```
str(10)          # Converts integer 10 to string "10"
print("Random " + str(10))    # Works!
```

String to Integer

```
int("22")        # Converts string "22" to integer 22
age = int(input("What is your age? "))
```

Important rule: You can only convert a string to int if the string contains a valid base-10 number.

```
int("hello")     # ValueError! "hello" is not a number
int("22")        # Works fine
```

 **Instructor Note:** When the error says 'invalid literal for int() with base 10' — it means the string you're trying to convert doesn't look like a number. Base 10 is just the regular decimal system humans use (digits 0–9).

Summary of Type Conversions

str(x)	Convert x to string. Always works.
int(x)	Convert x to integer. x must be a valid decimal number string.
float(x)	Convert x to float. x must be a valid number string.
type(x)	Check the current data type of x.

8. Terminal & Command Line Basics

The instructor paused the Python lesson to teach essential terminal commands, which are needed to navigate and work with files.

Key Commands

ls	List all files and folders in the current directory.
pwd	Print Working Directory – shows which folder you're currently in.
cd foldername	Change Directory – go into a specific folder.
cd ..	Go one folder back (to the parent folder).
Tab key	Auto-complete folder/file names. Type first letter + Tab.

Important Tip

'Directory' and 'folder' mean the same thing. The term 'directory' is used in programming/terminal contexts.

9. VS Code Setup & Best Practices

The instructor walked through a recommended folder structure so students don't get confused later.

Steps to Set Up Your Workspace

- Open VS Code.
- Go to File → Open Folder.
- Go to Desktop. Click New Folder. Name it: python_classes (no spaces in folder names).
- Click Select Folder — this opens your workspace.
- Create a new file: click the new file icon → name it test.py
- Open the terminal inside VS Code: three-dot menu → Terminal → New Terminal.
- The terminal will open already pointing to your python_classes folder.

 **Instructor Note:** Always create a dedicated folder for your class files. It keeps your work organized and avoids path confusion when running files from the terminal. Also: never use spaces in Python file names. Use underscores instead (e.g., my_script.py).

10. Virtual Environments

A virtual environment is an isolated Python environment for your project. It keeps your project's packages separate from other projects.

Why Use Virtual Environments?

- Different projects may need different versions of the same package.
- A virtual environment keeps your project clean and isolated.
- It is a professional best practice — always use one for every project.

Commands

Create venv	<code>python -m venv venv</code> (creates a folder called 'venv')
Activate (Mac/Linux)	<code>source venv/bin/activate</code>
Activate (Windows)	<code>venv\Scripts\activate</code> OR <code>.\venv\Scripts\activate</code>
Deactivate	<code>deactivate</code>
Verify activated	Look for (venv) prefix in your terminal prompt

Windows Permission Issue

Some Windows machines block script execution by default. If you get an error when activating, you need to tell your system to allow it. Ask the error message to an AI tool — it will give you the exact steps to fix it.

💡 **Instructor Note:** Always activate your virtual environment *BEFORE* installing packages or running your project. If you install packages without activating, they go to the global Python installation, which can cause version conflicts.

11. Installing Packages – pip

pip is Python's package manager. It is used to install third-party libraries.

```
pip install requests
```

Always make sure your virtual environment is activated before running pip install, so packages are installed into your project environment only.

12. API Calls with Python – requests Library

The requests library is a Python package that allows you to make HTTP requests (API calls) from your Python code.

What is a Request?

- A request is an API call made from a client to a server.
- Client = anything that initiates the request (phone, laptop, CCTV, your Python script).
- Server = the destination that processes the request and returns data.

HTTP Methods (Quick Intro)

- GET – fetch/retrieve data (used today).
- POST – send/create data.
- PUT / PATCH – update data.
- DELETE – delete data.

We will have a dedicated class for HTTP methods. Today's focus is GET.

Public API – GitHub User Endpoint

The class used a real public GitHub API endpoint that requires no login or credentials:

```
https://api.github.com/users/{your_github_username}
```

Open this in your browser first — you should see a JSON response body with your GitHub profile data (name, username, number of followers, repositories, etc.).

 **Instructor Note:** This is the kind of API your applications will create and consume. Your end goal as a Python developer is to build similar endpoints for your own servers. GitHub's API is a great example to study.

Making the API Call in Python

Write this 3-line Python script:

```
import requests
url = "https://api.github.com/users/your_username"
response = requests.get(url)
```

Understanding the Response

- The response object has two main things you need right now:

```
response.status_code    # Gives a number (e.g., 200)
response.json()         # Gives the response body as a Python dictionary
```

What is Status Code 200?

When you make an API call, the server sends back:

- Response body – the actual data (in JSON format).
- Status code – a number indicating the result.
- Headers – metadata (we'll cover this in a later class).

200	OK – request was successful.
404	Not Found – the URL or resource doesn't exist.
401	Unauthorized – you need login credentials.
500	Internal Server Error – something went wrong on the server.

JSON vs Python Dictionary

The response body from an API is in JSON format. Python does not natively understand JSON, but the requests library can automatically convert it to a Python dictionary:

```
data = response.json()    # Converts JSON to Python dict
print(data)               # Prints the full dictionary
```

To access a specific field from the response:

```
data = response.json()
print(data.get("login"))  # Your GitHub username
```

```
print(data.get("followers")) # Number of followers
```

💡 **Instructor Note:** JSON (JavaScript Object Notation) is the standard format for data exchange on the internet. It looks almost identical to a Python dictionary. When you call `response.json()`, requests converts it to a real Python dict so you can work with it using regular Python dictionary operations like `.get()`.

Full Example Code

```
import requests

url = "https://api.github.com/users/your_username"
response = requests.get(url)

print(response.status_code) # Should print 200
data = response.json()
print(data.get("login"))
print(data.get("name"))
print(data.get("followers"))
```

💡 **Instructor Note:** Remember to always save your file before running it! A common mistake is editing the code but forgetting to save, then running the old version and wondering why nothing changed.

13. Homework / Practice Task

The instructor gave the following task before winding up the class:

- Run the GitHub API call successfully.
- From the `response.json()` dictionary, pick any 5 fields that seem important to you (e.g., login, name, followers, public_repos, bio).
- Print each of those fields using `data.get("key_name")`.
- Practice accessing dictionary values using the key.

💡 **Instructor Note:** This task is important because it reinforces two concepts together: making an API call AND working with dictionaries. Both are core Python skills you'll use constantly in real projects.

14. Quick Reference – Key Concepts

print()	Prints output to the terminal.
input()	Takes user input. Always returns a string.
type(x)	Returns the data type of x.
str(x)	Converts x to string.
int(x)	Converts x to integer (must be valid number).
import requests	Imports the requests library for API calls.
requests.get(url)	Makes an HTTP GET request to the given URL.
response.status_code	Returns HTTP status code (200 = OK).
response.json()	Returns the response body as a Python dict.
dict.get(key)	Safely retrieves a value from a dictionary by key.
python file.py	Command to run a Python file from terminal.
python -m venv venv	Creates a virtual environment.
pip install pkg	Installs a Python package.

End of Day 3 Notes